# LAB 3

## Thực hành Tổng hợp

Fullstack Integration: NextJS + Express

| | |
|---|---|
| **Chủ đề** | Kết nối Frontend NextJS với Backend Express để quản lý giỏ hàng / sản phẩm |
| **Thời lượng** | 5 tiết |
| **Yêu cầu** | Node.js 18+, npm, VSCode, kiến thức JavaScript cơ bản |

# Tiết 1: Thiết lập môi trường & Xử lý CORS

Mục tiêu: Dựng xong 2 server chạy song song, giao tiếp được với nhau mà không bị lỗi CORS.

## Cấu trúc dự án

TERMINAL — KHỞI TẠO PROJECT

```
# Tạo thư mục gốc
mkdir fullstack-shop && cd fullstack-shop

# Backend
mkdir backend && cd backend
npm init -y
npm install express cors dotenv

# Frontend (mở tab terminal mới)
cd ..
npx create-next-app@latest frontend --typescript --tailwind --app
```

Cấu trúc thư mục sau khi khởi tạo:

```
fullstack-shop/
├── backend/
│   ├── server.js
│   └── package.json
└── frontend/
    ├── app/
    ├── lib/
    └── package.json
```

## Bước 1 — Dựng Backend Express cơ bản

BACKEND/SERVER.JS

```
const express = require('express');
const cors    = require('cors');
const app     = express();

app.use(cors({
  origin: 'http://localhost:3000',  // chỉ cho phép NextJS
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type']
}));
app.use(express.json());

let products = [
  { id: 1, name: 'Áo thun basic', price: 150000 },
  { id: 2, name: 'Quần jeans slim', price: 450000 },
];

app.get('/api/products', (req, res) => {
  res.json(products);
});

app.listen(5000, () =>
```

```
  console.log('Backend chạy tại port :5000')
);
```

**Mẹo:** Chạy backend: node server.js — giữ terminal này mở suốt buổi thực hành.

## Bước 2 — Cố tình tái hiện lỗi CORS

Thực hiện theo thứ tự để hiểu bản chất lỗi CORS:

1. Tạm thời xoá dòng app.use(cors(...)) trong server.js, restart backend.
2. Từ NextJS gọi fetch('http://localhost:5000/api/products'), mở DevTools → Console → quan sát thông báo lỗi.
3. Chụp màn hình lỗi. Ghi nhận: trình duyệt chặn ở phía nào (client hay server)?
4. Khôi phục lại middleware cors(), reload trang — lỗi biến mất.

**Cảnh báo:** Lỗi CORS xảy ra ở trình duyệt (browser), KHÔNG phải ở server hay terminal. Header Access-Control-Allow-Origin phải do server gửi về.

## Bước 3 — Cấu hình Proxy trong NextJS (cách thay thế)

NEXT.CONFIG.TS

```
const nextConfig = {
  async rewrites() {
    return [{
      source: '/api/:path*',
      destination: 'http://localhost:5000/api/:path*'
    }];
  }
};
export default nextConfig;
```

**Lưu ý:** Với proxy, frontend gọi /api/products (cùng origin) → NextJS tự chuyển tiếp sang backend. Không cần cors() trên Express nữa. Đây là cách phổ biến trong môi trường production.

# Tiết 2: Kết nối Form → API Backend

Mục tiêu: Tạo form thêm sản phẩm ở Frontend, gửi dữ liệu lên POST /api/products bằng fetch và axios.

## Bước 4 — Thêm route POST trên Backend

BACKEND/SERVER.JS — THÊM VÀO CUỐI FILE

```
app.post('/api/products', (req, res) => {
  const { name, price } = req.body;

  // Validation đơn giản
  if (!name || !price)
    return res.status(400).json({ error: 'Thiếu dữ liệu' });

  const newProduct = {
    id: Date.now(),
    name,
    price: Number(price)
  };
  products.push(newProduct);
  res.status(201).json(newProduct);
});
```

## Bước 5 — Form với fetch thuần

FRONTEND/APP/PRODUCTS/PAGE.TSX

```
'use client';
import { useState, useEffect } from 'react';

export default function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [name, setName]         = useState('');
  const [price, setPrice]       = useState('');

  const fetchProducts = async () => {
    const res  = await fetch('http://localhost:5000/api/products');
    const data = await res.json();
    setProducts(data);
  };

  useEffect(() => { fetchProducts(); }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch('http://localhost:5000/api/products', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, price }),
    });
    if (res.ok) {
      setName(''); setPrice('');
      fetchProducts();   // refresh danh sách
    }
  };
```

```
  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input value={name}  onChange={e => setName(e.target.value)}
placeholder='Tên SP' />
        <input value={price} onChange={e => setPrice(e.target.value)}
placeholder='Giá' type='number' />
        <button type='submit'>Thêm sản phẩm</button>
      </form>
      {products.map(p => <div key={p.id}>{p.name} —
{p.price.toLocaleString()}đ</div>)}
    </div>
  );
}
```

## Bước 6 — Nâng cấp dùng axios

CÀI ĐẶT AXIOS

```
npm install axios
```

FRONTEND/LIB/API.TS — FILE CẤU HÌNH TRUNG TÂM

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000',
  headers: { 'Content-Type': 'application/json' },
});

export default api;
```

DÙNG TRONG COMPONENT

```
import api from '@/lib/api';

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await api.post('/api/products', { name, price });
    fetchProducts();
  } catch (err) {
    console.error(err.response?.data?.error);
  }
};
```

> **Lưu ý:** Ưu điểm axios: tự parse JSON, xử lý error dễ hơn qua err.response, dễ thêm interceptor (auth token, logging) sau này. Nên dùng axios cho dự án thực tế.

# Tiết 3: Toast Message & Debug

Mục tiêu: Hiển thị thông báo phản hồi người dùng và biết cách debug lỗi giữa 2 phía.

## Bước 7 — Tích hợp Toast với react-hot-toast

CÀI ĐẶT

```
npm install react-hot-toast
```

FRONTEND/APP/LAYOUT.TSX — THÊM TOASTER VÀO ROOT LAYOUT

```
import { Toaster } from 'react-hot-toast';

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        {children}
        <Toaster position='top-right' />
      </body>
    </html>
  );
}
```

CÁC LOẠI TOAST SỬ DỤNG TRONG COMPONENT

```
import toast from 'react-hot-toast';

// Toast thành công
toast.success('Thêm sản phẩm thành công!');

// Toast xoá
toast.success('Đã xoá sản phẩm', { icon: '🗑' });

// Toast lỗi
toast.error('Không thể kết nối server!');

// Async toast: loading → success/error tự động
toast.promise(
  api.post('/api/products', data),
  {
    loading: 'Đang lưu...',
    success: 'Lưu thành công!',
    error:   'Có lỗi xảy ra!',
  }
);
```

**Mẹo:** toast.promise() là cách tốt nhất cho thao tác async: tự hiện loading, rồi chuyển sang success/error khi promise resolve/reject.

## Bước 8 — Debug lỗi hệ thống

### Phía Frontend — Browser DevTools

1. DevTools → tab Network: xem request đã gửi đi chưa, status code bao nhiêu.
2. Tab Headers: kiểm tra request headers, method, payload đã đúng chưa.
3. Tab Response: đọc message lỗi server trả về.
4. Tab Console: xem lỗi JavaScript, uncaught promise rejection.

### Phía Backend — Terminal Express

- Thêm console.log(req.body) đầu mỗi route để xác nhận data nhận được.
- Thêm middleware log tổng quát: app.use((req, _, next) => { console.log(req.method, req.url); next() })
- Kiểm tra port đang chạy: lsof -i :5000 (macOS/Linux) hoặc netstat -ano | findstr :5000 (Windows).
- Status code: 400 = lỗi từ client gửi sai, 500 = lỗi trong code server.

### Bảng lỗi thường gặp

| Mã lỗi / Thông báo | Nguyên nhân & Hướng xử lý |
|---|---|
| `CORS error` | Thiếu middleware cors() hoặc sai origin trong allowedOrigins |
| `404 Not Found` | Sai URL, sai HTTP method (GET thay vì POST) |
| `400 Bad Request` | Body rỗng hoặc thiếu header Content-Type: application/json |
| `500 Internal Error` | Bug trong code Express, kiểm tra terminal backend |
| `ERR_CONNECTION_REFUSED` | Backend chưa chạy hoặc cấu hình sai port |

# Tiết 4-5: Bài tập tự làm & Phần nâng cao

Mục tiêu: Hoàn thiện chức năng Xóa sản phẩm end-to-end và tự động cập nhật danh sách.

## Bài tập bắt buộc: Xóa sản phẩm end-to-end

### Phần 1 — Backend: thêm route DELETE

BACKEND/SERVER.JS — THÊM VÀO CUỐI FILE

```
app.delete('/api/products/:id', (req, res) => {
  const id    = Number(req.params.id);
  const index = products.findIndex(p => p.id === id);

  if (index === -1)
    return res.status(404).json({ error: 'Không tìm thấy sản phẩm' });

  products.splice(index, 1);
  res.json({ message: 'Đã xoá thành công' });
});
```

### Phần 2 — Frontend: hàm xoá + revalidate

THÊM VÀO COMPONENT PRODUCTSPAGE

```
const handleDelete = async (id: number) => {
  // 1. Xác nhận người dùng
  if (!confirm('Bạn chắc chắn muốn xoá?')) return;

  try {
    // 2. Gọi API xoá
    await api.delete(`/api/products/${id}`);

    // 3. Cập nhật state NGAY (optimistic update)
    //    — không cần gọi lại API, UX nhanh hơn
    setProducts(prev => prev.filter(p => p.id !== id));

    // 4. Hiển thị toast thành công
    toast.success('Đã xoá sản phẩm');

  } catch (err) {
    toast.error('Xoá thất bại, thử lại!');
    // Rollback: gọi lại server để đồng bộ dữ liệu
    fetchProducts();
  }
};
```

### Phần 3 — UI: nút Xoá trong danh sách

```
{products.map(p => (
  <div key={p.id} className='flex justify-between items-center p-3 border rounded
mb-2'>
    <span>
      {p.name} — {p.price.toLocaleString()}đ
```

```
    </span>
    <button
      onClick={() => handleDelete(p.id)}
      className='text-red-500 hover:text-red-700 text-sm font-medium'
    >
      Xoá
    </button>
  </div>
))}
```

**Mẹo:** Kiểm tra: sau khi nhấn Xoá và xác nhận, sản phẩm biến mất khỏi danh sách NGAY LẬP TỨC mà không cần F5 trang. Đây là optimistic update.

## Checklist tự kiểm tra trước khi nộp bài

| | |
|---|---|
| [ ] | Backend chạy tại port 5000, có đủ 3 route: GET / POST / DELETE /api/products |
| [ ] | Frontend gọi API thành công, không còn lỗi CORS trong Console |
| [ ] | Form thêm sản phẩm hoạt động, dữ liệu mới xuất hiện trong danh sách |
| [ ] | Nút Xoá hoạt động, danh sách tự cập nhật sau khi xoá (không cần F5) |
| [ ] | Toast hiện khi thêm thành công, xoá thành công, và khi có lỗi |
| [ ] | Không có lỗi đỏ trong Console (cả browser và terminal backend) |

# Phần Nâng Cao (BONUS)

Hoàn thành bài tập cơ bản trước. Phần nâng cao để đạt điểm xuất sắc và chuẩn bị cho dự án thực tế.

## Nâng cao 1 — Chỉnh sửa sản phẩm (PUT)

Thêm route PUT /api/products/:id trên backend để cập nhật thông tin sản phẩm:

```
app.put('/api/products/:id', (req, res) => {
  const id    = Number(req.params.id);
  const index = products.findIndex(p => p.id === id);
  if (index === -1)
    return res.status(404).json({ error: 'Không tìm thấy' });

  products[index] = { ...products[index], ...req.body };
  res.json(products[index]);
});
```

Trên frontend: thêm nút Sửa ở mỗi sản phẩm. Khi bấm, hiển thị inline form (hoặc modal) với giá trị hiện tại. Gửi PUT request khi submit, cập nhật lại danh sách.

## Nâng cao 2 — React Query (TanStack Query) ★ Khuyến khích

> **Lưu ý:** Đây là pattern chuẩn trong dự án thực tế. Nên ưu tiên hoàn thành mục này.

```
npm install @tanstack/react-query
```

Thay useEffect + fetch thủ công bằng useQuery và useMutation:

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

const queryClient = useQueryClient();

// Fetch danh sách — tự cache & refetch
const { data: products, isLoading } = useQuery({
  queryKey: ['products'],
  queryFn: () => api.get('/api/products').then(r => r.data)
});

// Mutation xoá — tự invalidate cache sau khi thành công
const deleteMutation = useMutation({
  mutationFn: (id) => api.delete(`/api/products/${id}`),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['products'] });
    toast.success('Đã xoá sản phẩm');
  },
  onError: () => toast.error('Xoá thất bại!'),
});
```

Mục tiêu hiểu: caching tự động, invalidateQueries để revalidate, staleTime để kiểm soát tần suất refetch.

## Nâng cao 3 — Lưu dữ liệu vào file JSON

Thay mảng products trong RAM bằng đọc/ghi file data.json dùng fs.promises. Dữ liệu không mất khi restart server:

```
const fs   = require('fs').promises;
const PATH = './data.json';

async function readData() {
  const raw = await fs.readFile(PATH, 'utf-8');
  return JSON.parse(raw);
}

async function writeData(data) {
  await fs.writeFile(PATH, JSON.stringify(data, null, 2));
}

// Dùng trong route:
app.get('/api/products', async (req, res) => {
  const products = await readData();
  res.json(products);
});
```

## Nâng cao 4 — Chức năng Giỏ hàng (Cart)

Xây dựng thêm module Cart hoàn chỉnh:

- Backend: thêm route POST /api/cart nhận { productId, quantity }, GET /api/cart, DELETE /api/cart/:productId
- Frontend: nút Thêm vào giỏ ở mỗi sản phẩm
- Hiển thị badge số lượng item trên icon giỏ hàng (cập nhật real-time)
- Trang /cart: danh sách sản phẩm trong giỏ, tổng tiền, nút xoá từng item

> **Lưu ý:** Nâng cao 2 (React Query) được khuyến khích nhất vì đây là pattern chuẩn trong môi trường production, giúp bạn hiểu sâu về data fetching và cache management.